Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Методы решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Пасевич К.Ю.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда; научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Ход работы

Вариант 9.

Формула ряда	x(k) = a·cos(b·k) + c·sin(d·k), k = 0…199
a	0.9
b	0.33
c	0.45
d	0.75
p (размер окна)	7

Итоговый ряд:
x(k) = 0.9·cos(0.33·k) + 0.45·sin(0.75·k), k = 0, 1, …, 199.
Обучающий участок: k = 0…139 (140 точек, 70 %). Тестовый участок: k = 140…199 (60 точек, 30 %).

Нормализация и обратный пересчёт
Нормализация Min-Max выполняется по обучающему участку:
x_norm = (x - x_min) / (x_max - x_min),
где x_min и x_max - минимум и максимум ряда на обучающем участке (k = 0…139).
Обратный пересчёт:
x = x_norm · (x_max - x_min) + x_min.
Для варианта 9 получено: x_min = -1.338467, x_max = 1.347060, размах x_max - x_min = 2.685527.
Почему min/max нельзя брать по всему ряду. Нормализация по всему ряду означала бы использование информации из тестового участка при подготовке данных, это утечка целевой информации (data leakage), которая завышает оценку качества и делает эксперимент некорректным. Кроме того, тестовые значения могут выходить за границы [0; 1], что при сигмоидных активациях скрытого слоя приводит к насыщению нейронов и ухудшению обучения. Поэтому все метрики в исходной шкале вычисляются через обратный пересчёт с параметрами, найденными только на train.

Схема формирования окон (скользящее окно)
Для каждого целевого момента k формируется входной вектор:
X = [x(k-1), x(k-2), …, x(k-p)],
цель d = x(k). Окно «скользит» по ряду, давая по одному примеру на каждую допустимую позицию.
Количество примеров:

- train: k = p…139 → 140 - p примеров; при p = 7 - 133 примера;
- test: k = 140…199 → 60 примеров (при любом p ≤ 140).

Архитектура p–N–1

- Входной слой: p нейронов (нормализованные значения окна).
- Скрытый слой: N нейронов с сигмоидой: h_j = σ(s_j) = 1 / (1 + e^(−s_j)), s_j = Σ_i W1_ji·x_i + b1_j.
- Выходной слой: 1 нейрон, линейный: y = Σ_j W2_j·h_j + b2.
- Функция потерь: MSE = (1/M)·Σ (y − d)².

Вывод градиентов (обратное распространение)
Обозначим δ_out = y − d (производная MSE по выходу с учётом множителя 2, который поглощается скоростью обучения).
Выходной линейный слой. Так как y = W2·h + b2, то:
∂E/∂W2_j = δ_out · h_j,
∂E/∂b2 = δ_out.
Скрытый сигмоидный слой. Сигмоида имеет производную σ′(s) = σ(s)(1 − σ(s)) = h_j(1 − h_j). Тогда:

$\delta_{hj} = \delta_{out} \cdot W2_j \cdot h_j \cdot (1 - h_j)$,
$\partial E/\partial W1_{ji} = \delta_{hj} \cdot x_i$,
$\partial E/\partial b1_j = \delta_{hj}$.
Обновление весов (онлайн-режим, по одному примеру): $W \leftarrow W - \eta \cdot \partial E/\partial W$.

Формулы метрик

- $MAE = (1/n) \cdot \Sigma |y_i - \hat{y}_i|$;
- $RMSE = \sqrt{[(1/n) \cdot \Sigma (y_i - \hat{y}_i)^2]}$;
- $MAPE = (100/n) \cdot \Sigma |(y_i - \hat{y}_i)/y_i|$;
- $sMAPE = (100/n) \cdot \Sigma 2 \cdot |y_i - \hat{y}_i| / (|y_i| + |\hat{y}_i|)$.

Код программы:

```
import sys
import json
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from multiprocessing import Pool

A, B, C, D = 0.9, 0.33, 0.45, 0.75
P_VAR, N_VAR = 7, 4

N_TOTAL = 200
N_TRAIN = 140
EE = 1e-4
MAX_EPOCHS = 20000
ETA = 0.1
BASE_SEED = 42
SEEDS = [1, 2, 3, 4, 5]


def generate_series(n=N_TOTAL):
    k = np.arange(n)
    return A * np.cos(B * k) + C * np.sin(D * k)


def fit_minmax(train_part):
    return               float(np.min(train_part)),
float(np.max(train_part))


def normalize(x, xmin, xmax):
    return (x - xmin) / (xmax - xmin)


def denormalize(xn, xmin, xmax):
    return xn * (xmax - xmin) + xmin


def    make_windows(series_n,    p,    first_target,
last_target):
    X, d = [], []
    for k in range(first_target, last_target + 1):
        X.append([series_n[k - j] for j in range(1,
p + 1)])
        d.append(series_n[k])
    return np.array(X), np.array(d)


def sigmoid(s):
    return 1.0 / (1.0 + np.exp(-s))


def init_weights(p, n_hidden, seed):
    rng = np.random.default_rng(seed)
    return {
        "W1":  rng.uniform(-0.5,  0.5,  (n_hidden,
p)),
        "b1": rng.uniform(-0.5, 0.5, n_hidden),
        "W2": rng.uniform(-0.5, 0.5, n_hidden),
        "b2": float(rng.uniform(-0.5, 0.5)),
    }


def forward(w, x):
    h = sigmoid(w["W1"] @ x + w["b1"])
    y = float(w["W2"] @ h + w["b2"])
    return h, y


def predict_batch(w, X):
    H = sigmoid(X @ w["W1"].T + w["b1"])
    return H @ w["W2"] + w["b2"]


def backward(w, x, h, y, d):
    delta_out = y - d
    g_W2 = delta_out * h
    g_b2 = delta_out
    delta_h = delta_out * w["W2"] * h * (1.0 - h)
    g_W1 = np.outer(delta_h, x)
    g_b1 = delta_h
    return g_W1, g_b1, g_W2, g_b2


def mse(w, X, d):
    return float(np.mean((predict_batch(w, X)  - d)
** 2))


def  train(Xtr,  dtr,  p,  n_hidden,  seed,  eta=ETA,
ee=EE, max_epochs=MAX_EPOCHS):
    w = init_weights(p, n_hidden, seed)
    rng = np.random.default_rng(seed + 1000)
    M = len(dtr)
    history = []
    epochs = 0
    reached = False
    for ep in range(1, max_epochs + 1):
        for i in rng.permutation(M):
            x = Xtr[i]
            h, y = forward(w, x)
            gW1, gb1, gW2, gb2 = backward(w, x, h,
y, dtr[i])
            w["W1"] -= eta * gW1
            w["b1"] -= eta * gb1
            w["W2"] -= eta * gW2
            w["b2"] -= eta * gb2
        es = mse(w, Xtr, dtr)
        history.append(es)
        epochs = ep
        if es <= ee:
            reached = True
            break
    return w, history, epochs, reached


def mae(y, yh):
    return float(np.mean(np.abs(y - yh)))


def rmse(y, yh):
    return float(np.sqrt(np.mean((y - yh) ** 2)))


def mape(y, yh):
    return  float(np.mean(np.abs((y  -  yh)  /  y))  *
100)


def smape(y, yh):
```

```
    denom = np.abs(y) + np.abs(yh)
    eps = 1e-12
    return float(np.mean(2 * np.abs(y - yh) / (denom
+ eps)) * 100)


def all_metrics(y, yh):
    return {"MAE": mae(y, yh), "RMSE": rmse(y, yh),
"sMAPE": smape(y, yh)}


def prepare(series, p):
    xmin, xmax = fit_minmax(series[:N_TRAIN])
    sn = normalize(series, xmin, xmax)
    Xtr, dtr = make_windows(sn, p, p, N_TRAIN - 1)
    Xte, dte = make_windows(sn, p, N_TRAIN, N_TOTAL
- 1)
    return sn, xmin, xmax, Xtr, dtr, Xte, dte


def run_config(args):
    p, n_hidden, seed, eta, noise_sigma = args
    series = generate_series()
    sn, xmin, xmax, Xtr, dtr, Xte, dte =
prepare(series, p)

    if noise_sigma > 0:
        rng = np.random.default_rng(seed + 5000)
        sn_noisy = sn.copy()
        sn_noisy[:N_TRAIN]     +=     rng.normal(0,
noise_sigma, N_TRAIN)
        Xtr, dtr = make_windows(sn_noisy, p, p,
N_TRAIN - 1)

    w, hist, ep, ok = train(Xtr, dtr, p, n_hidden,
seed, eta=eta)

    Xtr_clean, dtr_clean = make_windows(sn, p, p,
N_TRAIN - 1)
    ytr = denormalize(dtr_clean, xmin, xmax)
    ptr = denormalize(predict_batch(w, Xtr_clean),
xmin, xmax)

    yte = denormalize(dte, xmin, xmax)
    pte = denormalize(predict_batch(w, Xte), xmin,
xmax)

    return {
        "p": p, "N": n_hidden, "seed": seed, "eta":
eta, "epochs": ep, "reached": ok,
        "Es": hist[-1], "train": all_metrics(ytr,
ptr), "test": all_metrics(yte, pte),
        "history": hist,
        "w": {k: (v.tolist() if hasattr(v, "tolist")
else v) for k, v in w.items()},
    }


def summarize(runs):
    ep = np.array([r["epochs"] for r in runs],
float)
    te = np.array([r["test"]["MAE"] for r in runs])
    return {
        "epochs_mean":            float(ep.mean()),
"epochs_std": float(ep.std(ddof=0)),
        "epochs_min":             float(ep.min()),
"epochs_max": float(ep.max()),
        "mae_mean":  float(te.mean()),  "mae_std":
float(te.std(ddof=0)),
        "mae_min":   float(te.min()),   "mae_max":
float(te.max()),
        "failed": int(sum(not r["reached"] for r in
runs)),
    }


def recursive_forecast(w, series_n, p, start,
steps):
    window = [series_n[start - j] for j in range(1,
p + 1)]
    preds = []
    wn = {k: np.array(v) for k, v in w.items()}
    for _ in range(steps):
        _, y = forward(wn, np.array(window))
        preds.append(y)
        window = [y] + window[:-1]
    return np.array(preds)


def functioning_mode(w, p, xmin, xmax, series,
window_raw=None, k=None):
    wn = {kk: np.array(v) for kk, v in w.items()}
    truth = None
    if k is not None:
        window_raw = [series[k - j] for j in
range(1, p + 1)]
        truth = series[k]
    window_raw = np.array(window_raw, float)
    xin = normalize(window_raw, xmin, xmax)
    _, yn = forward(wn, xin)
    y = denormalize(yn, xmin, xmax)
    return {
        "window_raw": window_raw.tolist(),
        "inputs_norm": xin.tolist(),
        "pred_norm": float(yn),
        "pred": float(y),
        "truth":  None  if  truth  is  None  else
float(truth),
        "abs_err":  None  if  truth  is  None  else
float(abs(truth - y)),
        "k": k,
    }


def format_func(res):
    s = []
    if res["k"] is not None:
        s.append(f"   window from series for k =
{res['k']}")
    s.append("  input (original scale, order x_{k-
1}..x_{k-p}): " +
             ",   ".join(f"{v:.4f}"   for   v   in
res["window_raw"]))
    s.append("  normalized inputs: " +
             ",   ".join(f"{v:.4f}"   for   v   in
res["inputs_norm"]))
    s.append(f"     forecast   (normalized   scale):
{res['pred_norm']:.4f}")
    s.append(f"      forecast   (original   scale):
{res['pred']:.4f}")
    if res["truth"] is not None:
        s.append(f"  true x(k) = {res['truth']:.4f},
|error| = {res['abs_err']:.4f}")
    return "\n".join(s)


def main():
    series = generate_series()
    k_all = np.arange(N_TOTAL)
    xmin, xmax = fit_minmax(series[:N_TRAIN])

    out = {
        "xmin": xmin, "xmax": xmax,
        "range_full":     float(series.max()     -
series.min()),
        "full_min":            float(series.min()),
"full_max": float(series.max()),
    }

    sn_all = normalize(series, xmin, xmax)
    out["test_norm_min"]                           =
float(sn_all[N_TRAIN:].min())
    out["test_norm_max"]                           =
float(sn_all[N_TRAIN:].max())
    out["train_norm_min"]                          =
float(sn_all[:N_TRAIN].min())
    out["train_norm_max"]                          =
float(sn_all[:N_TRAIN].max())
    out["min_abs_series"]                          =
float(np.min(np.abs(series)))

    plt.figure(figsize=(10, 4))
    plt.plot(k_all,    series,    "b-",    lw=1.4,
label="x(k)")
    plt.axvline(N_TRAIN - 0.5, color="r", ls="--",
label="train/test boundary")
    plt.axvspan(0,  N_TRAIN  -  0.5,  color="green",
alpha=0.07)
```

```
    plt.axvspan(N_TRAIN - 0.5, N_TOTAL,
color="orange", alpha=0.10)
    plt.xlabel("k")
    plt.ylabel("x(k)")
    plt.title("Series x_k = 0.9*cos(0.33k) +
0.45*sin(0.75k) (variant 9)")
    plt.legend()
    plt.grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig("fig0_series.png", dpi=150)
    plt.close()

    eta_res = []
    with Pool() as pool:
        jobs = [(P_VAR, N_VAR, BASE_SEED, e, 0.0)
                for e in (0.01, 0.03, 0.05, 0.1,
0.2, 0.3)]
        for r in pool.map(run_config, jobs):
            eta_res.append({"eta": r["eta"],
"epochs": r["epochs"],
                            "reached":
r["reached"], "Es": r["Es"],
                            "test_MAE":
r["test"]["MAE"]})
    out["eta_sweep"] = eta_res
    print("eta-sweep:", eta_res, flush=True)

    base = run_config((P_VAR, N_VAR, BASE_SEED, ETA,
0.0))
    out["base"] = {kk: base[kk] for kk in
                   ("p", "N", "seed", "eta",
"epochs", "reached", "Es", "train", "test")}

    sn, xmin_b, xmax_b, Xtr_b, dtr_b, Xte_b, dte_b
= prepare(series, P_VAR)
    out["n_train_examples_p"] = int(len(dtr_b))
    out["n_test_examples_p"] = int(len(dte_b))
    print("base:", out["base"], flush=True)

    p_list = [2, 4, P_VAR, 2 * P_VAR]
    n_list = [1, 2, N_VAR, 2 * N_VAR]

    jobs_p = [(p, N_VAR, s, ETA, 0.0) for p in p_list
for s in SEEDS]
    jobs_n = [(P_VAR, n, s, ETA, 0.0) for n in n_list
for s in SEEDS]

    with Pool() as pool:
        res_all = pool.map(run_config, jobs_p +
jobs_n)
    res_p = res_all[:len(jobs_p)]
    res_n = res_all[len(jobs_p):]

    out["window_study"], out["hidden_study"] = {},
{}
    for i, p in enumerate(p_list):
        rs = res_p[i * 5:(i + 1) * 5]
        out["window_study"][str(p)] = {
            "summary": summarize(rs),
            "runs": [{"seed": r["seed"], "epochs":
r["epochs"],
                      "reached": r["reached"],
"Es": r["Es"],
                      "train_MAE":
r["train"]["MAE"],
                      "test_MAE": r["test"]["MAE"]}
for r in rs],
            "train_mae_mean":
float(np.mean([r["train"]["MAE"] for r in rs])),
        }
    for i, n in enumerate(n_list):
        rs = res_n[i * 5:(i + 1) * 5]
        out["hidden_study"][str(n)] = {
            "summary": summarize(rs),
            "runs": [{"seed": r["seed"], "epochs":
r["epochs"],
                      "reached": r["reached"],
"Es": r["Es"],
                      "train_MAE":
r["train"]["MAE"],
                      "test_MAE": r["test"]["MAE"]}
for r in rs],
            "train_mae_mean":
float(np.mean([r["train"]["MAE"] for r in rs])),
        }

    print(json.dumps({
        "window": {k: v["summary"] for k, v in
out["window_study"].items()},
        "hidden": {k: v["summary"] for k, v in
out["hidden_study"].items()},
    }, indent=1), flush=True)

    cand = res_p + res_n
    mean_by_cfg = {}
    for r in cand:
        mean_by_cfg.setdefault((r["p"], r["N"]),
[]).append(r)
    best_cfg = min(mean_by_cfg,
                   key=lambda c:
np.mean([r["test"]["MAE"] for r in
mean_by_cfg[c]]))
    best_run = min(mean_by_cfg[best_cfg],
key=lambda r: r["test"]["MAE"])
    out["best_cfg"] = {
        "p": best_cfg[0], "N": best_cfg[1], "seed":
best_run["seed"],
        "epochs": best_run["epochs"], "reached":
best_run["reached"],
        "test": best_run["test"], "train":
best_run["train"],
        "Es": best_run["Es"],
    }

    pb, nb = best_cfg
    snb, xminb, xmaxb, Xtrb, dtrb, Xteb, dteb =
prepare(series, pb)
    wb = best_run["w"]

    steps = 30
    rec_n = recursive_forecast(wb, snb, pb, N_TRAIN,
steps)
    rec = denormalize(rec_n, xminb, xmaxb)
    ref = series[N_TRAIN:N_TRAIN + steps]
    err = np.abs(ref - rec)
    thr = 0.05 * (series.max() - series.min())
    over = np.where(err > thr)[0]

    out["recursive"] = {
        "p": pb, "N": nb, "threshold": float(thr),
        "first_exceed_step": int(over[0] + 1) if
len(over) else None,
        "err": err.tolist(),
        "max_err": float(err.max()), "mean_err":
float(err.mean()),
        "err_step1": float(err[0]), "err_step5":
float(err[4]),
        "err_step10": float(err[9]), "err_step20":
float(err[19]),
        "err_step30": float(err[29]),
        "mae_30": float(err.mean()),
    }
    one_step_n = predict_batch({kk: np.array(v) for
kk, v in wb.items()},
                               Xteb[:steps])
    one_step = denormalize(one_step_n, xminb,
xmaxb)
    out["recursive"]["onestep_mae_30"] =
float(np.mean(np.abs(ref - one_step)))
    out["recursive"]["onestep_max_30"] =
float(np.max(np.abs(ref - one_step)))

    sigma = 0.05
    with Pool() as pool:
        noisy = pool.map(run_config,
                         [(P_VAR, N_VAR, s, ETA,
sigma) for s in SEEDS])
    out["noise"] = {
        "sigma_norm": sigma,
        "epochs": [r["epochs"] for r in noisy],
        "reached": [r["reached"] for r in noisy],
        "Es": [r["Es"] for r in noisy],
        "train_MAE_mean":
float(np.mean([r["train"]["MAE"] for r in noisy])),
        "test_MAE_mean":
float(np.mean([r["test"]["MAE"] for r in noisy])),
        "test_MAE_std":
float(np.std([r["test"]["MAE"] for r in noisy])),
```

```
        "clean_test_MAE_mean":
out["window_study"][str(P_VAR)]["summary"]["mae_me
an"],
    }

    wbase = {kk: np.array(v) for kk, v in
base["w"].items()}
    snp, xmin_p, xmax_p, Xtr7, dtr7, Xte7, dte7 =
prepare(series, P_VAR)
    pr_tr = denormalize(predict_batch(wbase, Xtr7),
xmin_p, xmax_p)
    pr_te = denormalize(predict_batch(wbase, Xte7),
xmin_p, xmax_p)

    plt.figure(figsize=(11, 4.5))
    plt.plot(k_all,    series,    "b-",    lw=1.4,
label="reference x(k)")
    plt.plot(np.arange(P_VAR,   N_TRAIN),   pr_tr,
"g.", ms=4,
             label="network forecast (train)")
    plt.plot(np.arange(N_TRAIN,  N_TOTAL),  pr_te,
"r.", ms=4,
             label="network forecast (test)")
    plt.axvline(N_TRAIN - 0.5, color="k", ls="--",
label="train/test boundary")
    plt.xlabel("k"); plt.ylabel("x(k)")
    plt.title("One-step forecast, p=7, N=4 (variant
9)")
    plt.legend();              plt.grid(alpha=0.3);
plt.tight_layout()
    plt.savefig("fig1_onestep.png",       dpi=150);
plt.close()

    h2 = [r for r in res_p if r["p"] ==
2][0]["history"]
    h14 = [r for r in res_p if r["p"] == 2 *
P_VAR][0]["history"]

    plt.figure(figsize=(8, 4.5))
    plt.semilogy(np.arange(1, len(base["history"])
+ 1), base["history"],
                 label=f"base       p=7,       N=4
({len(base['history'])} epochs)")
    plt.semilogy(np.arange(1, len(h2) + 1), h2,
label=f"p=2 ({len(h2)} epochs)")
    plt.semilogy(np.arange(1, len(h14) + 1), h14,
label=f"p=14 ({len(h14)} epochs)")
    plt.axhline(EE, color="k", ls=":", label="Ee =
1e-4")
    plt.xscale("log")
    plt.xlabel("epoch (log scale)")
    plt.ylabel("Es (normalized scale)")
    plt.title("Training convergence")
    plt.legend();              plt.grid(alpha=0.3,
which="both"); plt.tight_layout()
    plt.savefig("fig2_convergence.png",   dpi=150);
plt.close()

    yte7 = series[N_TRAIN:]
    plt.figure(figsize=(5.2, 5))
    plt.scatter(yte7, pr_te, s=14, c="r")
    lo = min(yte7.min(), pr_te.min()) - 0.1
    hi = max(yte7.max(), pr_te.max()) + 0.1
    plt.plot([lo, hi], [lo, hi], "k--", label="y =
x")
    plt.xlabel("reference"); plt.ylabel("forecast")
    plt.title("Test    segment:    reference    vs
forecast")
    plt.legend();              plt.grid(alpha=0.3);
plt.tight_layout()
    plt.savefig("fig3_scatter.png",       dpi=150);
plt.close()

    fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
    steps_ax = np.arange(1, steps + 1)
    ax[0].plot(steps_ax,    ref,    "b-o",    ms=3,
label="reference")
    ax[0].plot(steps_ax,    rec,    "r-s",    ms=3,
label="recursive forecast")
    ax[0].set_xlabel("step"); ax[0].set_ylabel("x")
    ax[0].set_title(f"Recursive  forecast  (p={pb},
N={nb})")
    ax[0].legend(); ax[0].grid(alpha=0.3)
    ax[1].plot(steps_ax,    err,    "m-o",    ms=3,
label="|error|")
    ax[1].axhline(thr, color="k", ls="--",
                  label=f"5% of range = {thr:.3f}")
    ax[1].set_xlabel("step");
ax[1].set_ylabel("absolute error")
    ax[1].set_title("Error growth by step")
    ax[1].legend(); ax[1].grid(alpha=0.3)
    plt.tight_layout()
    plt.savefig("fig4_recursive.png",     dpi=150);
plt.close()

    demo = []
    lines = [
        "OPERATING MODE",
        "(base network p=7, N=4, seed=42, variant
9)",
        f"xmin = {xmin:.6f}, xmax = {xmax:.6f}",
        "",
    ]
    cases = [("train", 20), ("train", 60), ("train",
120),
             ("test", 145), ("test", 170), ("test",
195)]
    for tag, k in cases:
        r  =  functioning_mode(base["w"],  P_VAR,
xmin, xmax, series, k=k)
        demo.append(dict(tag=tag, **r))
        lines.append(f"[{tag}]")
        lines.append(format_func(r))
        lines.append("")

    custom1 = [series[150 - j] for j in range(1,
P_VAR + 1)]
    r = functioning_mode(base["w"], P_VAR, xmin,
xmax, series,
                         window_raw=custom1)
    r["truth"] = float(series[150])
    r["abs_err"]    =    float(abs(r["truth"]    -
r["pred"]))
    r["k"] = None
    demo.append(dict(tag="user    (from    series,
k=150)", **r))
    lines.append("[user  window  =  series  values
k=143..149]")
    lines.append(format_func(r)); lines.append("")

    custom2 = list(custom1)
    custom2[3] += 3.0
    r = functioning_mode(base["w"], P_VAR, xmin,
xmax, series,
                         window_raw=custom2)
    r["truth"] = float(series[150])
    r["abs_err"]    =    float(abs(r["truth"]    -
r["pred"]))
    r["k"] = None
    demo.append(dict(tag="user with outlier (+3.0
in x_{k-4})", **r))
    lines.append("[user window with outlier: x_{k-
4} increased by 3.0]")
    lines.append(format_func(r)); lines.append("")

    open("functioning_demo.txt",              "w",
encoding="utf-8").write("\n".join(lines))
    out["demo"] = demo

    out["base_history_len"] = len(base["history"])
    with open("results.json", "w", encoding="utf-
8") as f:
        json.dump(out,    f,    ensure_ascii=False,
indent=1)
    with      open("base_weights.json",      "w",
encoding="utf-8") as f:
        json.dump(base["w"], f)

    print("\n".join(lines))

    if "--interactive" in sys.argv:
        print("\nInteractive mode.")
        print("Enter  'k=<number>'  or  7  values
separated by spaces; q to quit")
        while True:
            s = input("> ").strip()
            if s.lower() == "q":
```

```
                break
            if s.startswith("k="):
                res  =  functioning_mode(base["w"],
P_VAR, xmin, xmax, series,

k=int(s[2:]))
            else:
                res  =  functioning_mode(base["w"],
P_VAR, xmin, xmax, series,
```

```
window_raw=[float(v) for v in s.split()])
            print(format_func(res))


if __name__ == "__main__":
    main()
```

Обоснование выбора скорости обучения. Из η-sweep (диапазон 0.01-0.3) выбрано η = 0.1: сходимость за 130 эпох, test MAE = 0.02238. Меньшие η (0.01-0.05) требуют 250-1250 эпох без выигрыша в качестве; η = 0.3 сходится за 48 эпох, но даёт чуть худшее обобщение (MAE = 0.02267) и менее устойчиво. η = 0.1 - компромисс между скоростью и качеством.

Результаты базового эксперимента:

Конфигурация: p = 7, N = 4, seed = 42, η = 0.1.

Показатель	Значение
Число эпох	130
Достигнут критерий Es ≤ 10^{-4}	да
Итоговая Es (норм.)	$9.41 \cdot 10^{-5}$
Train MAE / RMSE / sMAPE	0.02222 / 0.02605 / 8.55 %
Test MAE / RMSE / sMAPE	0.02238 / 0.02601 / 15.62 %

Ошибки на train и test практически совпадают (0.0222 против 0.0224), что свидетельствует о хорошей обобщающей способности и отсутствии переобучения. Более высокий sMAPE на тесте объясняется наличием значений ряда вблизи нуля, где относительная ошибка формально велика при малой абсолютной.

Исследование размера окна p (N = 4, 5 запусков)

p	Эпохи: среднее ± std	Эпохи: min-max	Test MAE: среднее ± std	Test MAE: min-max	Не достигли Ee
2	20000 ± 0	20000-20000	0.09818 ± 0.00728	0.0928-0.1124	5 из 5
4	2678.8 ± 179.0	2382-2879	0.02120 ± 0.00030	0.02069-0.02160	0
7	114.6 ± 11.3	105-134	0.02216 ± 0.00094	0.02079-0.02330	0
14	31.8 ± 5.2	27-41	0.02110 ± 0.00075	0.01969-0.02183	0

При p = 2 модель не в состоянии описать ряд из двух гармоник: за 20 000 эпох Es застревает на уровне ~$2 \cdot 10^{-3}$, ни один из 5 запусков не достиг Ee, тестовая ошибка ~0.098 (в 4-5 раз хуже). Уже при p = 4 точность выходит на плато (~0.021), при p = 7 и p = 14 ошибка практически не уменьшается (различия в пределах разброса по seed), но резко падает число эпох: 2679 → 115 → 32. Дальнейшее увеличение окна свыше 7 даёт выигрыш только в скорости обучения, но не в точности.

Исследование числа скрытых нейронов N (p = 7, 5 запусков)

N	Эпохи: среднее ± std	Эпохи: min-max	Test MAE: среднее ± std	Test MAE: min-max	Не достигли Ee
1	129.4 ± 7.1	123-139	0.02211 ± 0.00053	0.02127-0.02290	0
2	113.8 ± 14.0	101-139	0.02169 ± 0.00113	0.01965-0.02305	0
4	114.6 ± 11.3	105-134	0.02216 ± 0.00094	0.02079-0.02330	0
8	101.0 ± 3.8	96-107	0.02095 ± 0.00073	0.01969-0.02193	0

Все конфигурации успешно обучаются. С ростом N время сходимости незначительно сокращается (129 → 101 эпоха), тестовая ошибка слабо уменьшается (0.0221 → 0.0210). Переобучения не наблюдается:

train MAE (0.0214-0.0224) и test MAE (0.0210-0.0222) практически совпадают при всех N. Уже N = 1 достаточно для аппроксимации суммы двух гармоник на одношаговом горизонте; увеличение N даёт маргинальный выигрыш.
Лучшая конфигурация по среднему test MAE: p = 7, N = 8, seed = 5 (test MAE = 0.01969, 96 эпох). Она использована для рекурсивного прогноза.

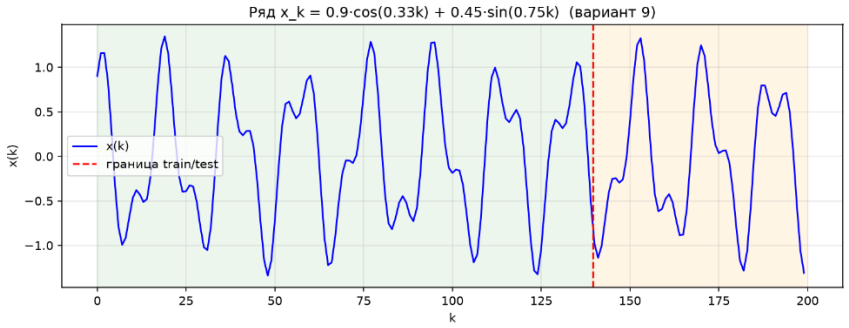
Многошаговый (рекурсивный) прогноз
Для лучшей конфигурации (p = 7, N = 8) выполнен рекурсивный прогноз на 30 шагов с k = 140: первое окно берётся из ряда (x_139…x_133), далее каждое предсказание подставляется в окно.
Ключевые результаты:

Показатель	Значение
Ошибка на шаге 1	0.0171
Ошибка на шаге 5	0.0145
Ошибка на шаге 10	0.2267
Ошибка на шаге 20	0.1211
Ошибка на шаге 30	0.2757
Средняя ошибка за 30 шагов	0.1461
Максимальная ошибка	0.4735 (шаг 26)
Порог 5 % размаха	0.1343
Первый шаг превышения порога	8
Одношаговый MAE за те же 30 точек	0.0208

На первых 6-7 шагах рекурсивная ошибка сопоставима с одношаговой (0.007-0.022), затем наступает лавинообразный рост: на 8-м шаге ошибка превышает 5 % размаха, к 26-му шагу достигает 0.47 (≈ 18 % размаха). Причина - накопление ошибки: каждое предсказание подаётся на вход следующего шага, и погрешность модели многократно усиливается, особенно в окрестностях экстремумов, где сигмоиды насыщены и чувствительность выхода максимальна. Одношаговый прогноз на том же участке остаётся точным (MAE = 0.021), что подтверждает: модель хорошо описывает локальную динамику, но не является генеративной моделью ряда.
Горизонт практической пригодности - примерно 7 шагов (до первого превышения порога 5 % размаха).

Визуализация:
В отчёте представлены четыре обязательных графика (файлы в каталоге проекта):

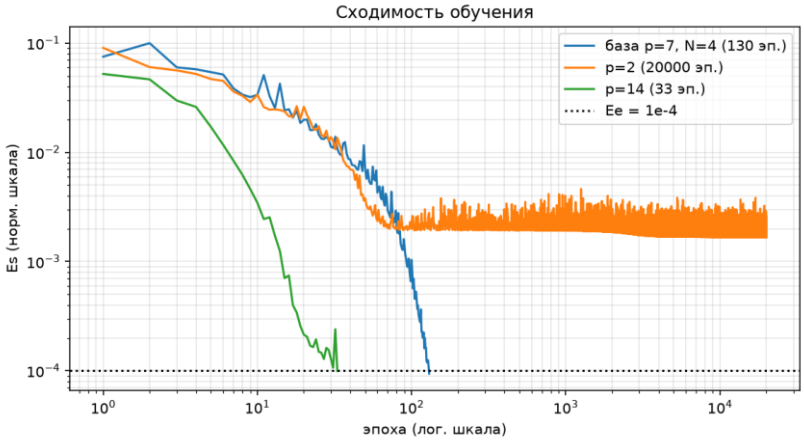
1. fig0_series.png - ряд x(k) с границей train/test (k = 139.5), обучающий участок выделен зелёным, тестовый - оранжевым.



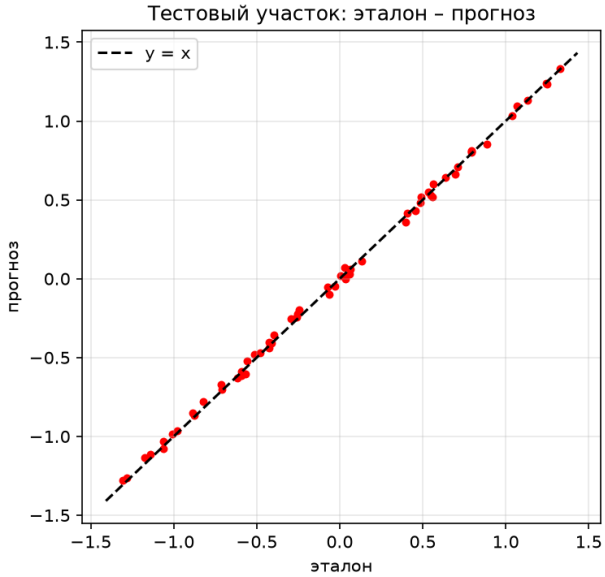
2. fig1_onestep.png - эталонный ряд и одношаговый прогноз сети на train (зелёные точки) и test (красные точки) на одних осях: прогноз практически совпадает с эталоном на обоих участках.



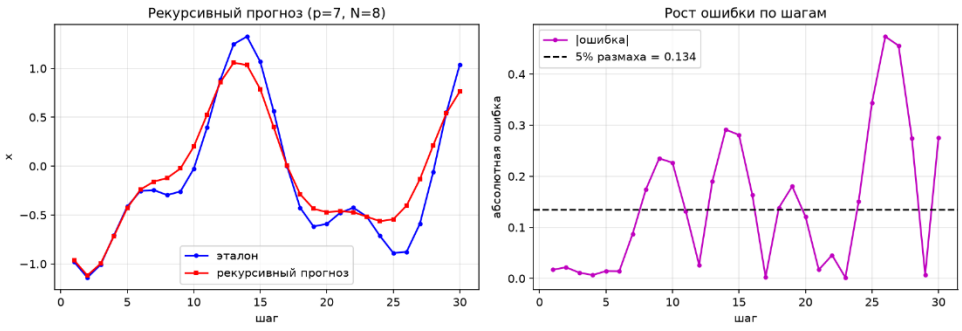
3. fig2_convergence.png - сходимость Es от эпохи (лог. шкала по X и Y) для базовой конфигурации p = 7 (130 эпох), p = 2 (20000 эпох, плато ~$2 \cdot 10^{-3}$, порог не достигнут) и p = 14 (33 эпохи); горизонтальная линия Ee = 10^{-4}.



4. fig3_scatter.png - диаграмма рассеяния «эталон - прогноз» на тестовом участке с диагональю y = x: точки плотно группируются вдоль диагонали, систематического смещения нет.



5. fig4_recursive.png - слева эталон и рекурсивный прогноз на 30 шагов, справа рост абсолютной ошибки по шагам с линией порога 5 % размаха.



Режим функционирования

Обучающий участок

1. k = 20. Вход (исх.): 1.3471, 1.2092, 0.7858, 0.2424, -0.2237, -0.4789, -0.5127. Норм. входы: 1.0000, 0.9487, 0.7910, 0.5887, 0.4151, 0.3201, 0.3075. Прогноз: 0.9323 → 1.1652. Истина 1.1478, |ошибка| = 0.0174.
2. k = 60. Вход: 0.8513, 0.6536, 0.4748, 0.4268, 0.5095, 0.6140, 0.5878. Норм.: 0.8154, 0.7418, 0.6752, 0.6573, 0.6881, 0.7271, 0.7173. Прогноз: 0.8356 → 0.9056. Истина 0.9061, |ошибка| = 0.0005.

3. k = 120. Вход: 0.4317, 0.5211, 0.4559, 0.3826, 0.4265, 0.6153, 0.8585. Норм.: 0.6592, 0.6925, 0.6682, 0.6409, 0.6572, 0.7275, 0.8181. Прогноз: 0.5524 → 0.1451. Истина 0.1106, |ошибка| = 0.0345.

Тестовый участок

4. k = 145. Вход: -0.4132, -0.7081, -1.0079, -1.1394, -0.9792, -0.5261, 0.0890. Норм.: 0.3445, 0.2347, 0.1231, 0.0741, 0.1338, 0.3025, 0.5316. Прогноз: 0.4161 → -0.2209. Истина -0.2526, |ошибка| = 0.0317.
5. k = 170. Вход: 1.0389, 0.5498, -0.0623, -0.5901, -0.8771, -0.8878, -0.7139. Норм.: 0.8852, 0.7031, 0.4752, 0.2786, 0.1718, 0.1678, 0.2326. Прогноз: 0.9605 → 1.2409. Истина 1.2452, |ошибка| = 0.0043.
6. k = 195. Вход: 0.7114, 0.6940, 0.5564, 0.4535, 0.4855, 0.6395, 0.7957. Норм.: 0.7633, 0.7568, 0.7056, 0.6673, 0.6792, 0.7365, 0.7947. Прогноз: 0.6916 → 0.5188. Истина 0.4912, |ошибка| = 0.0277.

Пользовательские окна

7. Окно из ряда k = 143…149 (цель k = 150). Вход: -0.0270, -0.2587, -0.2958, -0.2450, -0.2526, -0.4132, -0.7081. Норм.: 0.4884, 0.4021, 0.3883, 0.4072, 0.4043, 0.3445, 0.2347. Прогноз: 0.6324 → 0.3598. Истина 0.3958, |ошибка| = 0.0361.
8. То же окно с выбросом (x_{k-4} увеличен на +3.0): вход -0.0270, -0.2587, -0.2958, 2.7550, -0.2526, -0.4132, -0.7081. Норм.: 0.4884, 0.4021, 0.3883, 1.5243, 0.4043, 0.3445, 0.2347. Прогноз: 0.5231 → 0.0664. Истина 0.3958, |ошибка| = 0.3295.

Рукописные расчёты

(а) Схема пайплайна:

Генерация ряда → Train/Test split → Min-Max train → Sliding window → Normalize (по train) → MLP p-N-1 (скрытый слой - Sigmoid, выходной - линейный) → Online backprop → Shuffle after every epoch → Stop: $Es \le 10^{-4}$ → Denormalization → метрики (MAE, RMSE, sMAPE) → Recursive forecast → режим функционирования.

(б) Ручной расчёт одного шага обучения для сети 3-2-1.

Входы X = [0.2, 0.5, 0.8], цель d = 0.6.

Скрытый слой:

- W11 = 0.05·9 = 0.45; W12 = -0.2; W21 = 0.3; W22 = 0.05·9 = 0.45; W31 = -0.1; W32 = 0.4; b1 = 0.1; b2 = -0.1.
- s1 = 0.45·0.2 + (-0.2)·0.5 + 0.3·0.8 + 0.1 = 0.09 - 0.10 + 0.24 + 0.10 = 0.33; h1 = σ(0.33) ≈ 0.5818.
- s2 = (-0.1)·0.2 + 0.45·0.5 + 0.4·0.8 + (-0.1) = -0.02 + 0.225 + 0.32 - 0.10 = 0.425; h2 = σ(0.425) ≈ 0.6047.

Выходной слой:

- W_out1 = 0.5; W_out2 = -0.2·9 = -1.8; b_out = 0.2.
- y = 0.5·0.5818 + (-1.8)·0.6047 + 0.2 = 0.2909 - 1.0885 + 0.2 = -0.5976.

Ошибка: δ_out = y - d = -0.5976 - 0.6 = -1.1976.

Градиенты выходного слоя:

- ∂E/∂W_out1 = δ_out·h1 = -1.1976·0.5818 ≈ -0.6968;
- ∂E/∂W_out2 = δ_out·h2 = -1.1976·0.6047 ≈ -0.7242;
- ∂E/∂b_out = -1.1976.

Градиенты скрытого слоя:

- δ_h1 = δ_out·W_out1·h1(1-h1) = -1.1976·0.5·0.5818·0.4182 ≈ -0.1457;
- δ_h2 = δ_out·W_out2·h2(1-h2) = -1.1976·(-1.8)·0.6047·0.3953 ≈ +0.5151.
- ∂E/∂W11 = δ_h1·x1 = -0.1457·0.2 ≈ -0.0291; ∂E/∂W12 = δ_h1·x2 ≈ -0.0729; ∂E/∂W21 = δ_h1·x3 ≈ -0.1166;
- ∂E/∂W31 = δ_h2·x1 ≈ +0.1030; ∂E/∂W32 = δ_h2·x2 ≈ +0.2576; ∂E/∂W22 = δ_h2·x3 ≈ +0.4121;
- ∂E/∂b1 = δ_h1 ≈ -0.1457; ∂E/∂b2 = δ_h2 ≈ +0.5151.

При η = 0.1 новые веса: W11 ← 0.45 - 0.1·(-0.0291) = 0.4529; W_out1 ← 0.5 + 0.0697 = 0.5697; и т. д. Один шаг уменьшает ошибку: y_new ≈ -0.564, |y - d| снижается с 1.198 до ~1.164, что подтверждает корректность формул градиентов.

Ответы на вопросы

1. Влияние размера окна p. С ростом p от 2 до 4 ошибка резко падает (0.098 → 0.021), при p = 7 и p = 14 выходит на плато (0.021-0.022), ошибка перестаёт уменьшаться уже при p ≈ 4-7. Окно p = 2 физически недостаточно: ряд содержит две гармоники с периодами ≈ 19 и ≈ 8.4 отсчёта, и по двум точкам нельзя восстановить ни фазу, ни амплитуду, сеть видит лишь локальный наклон. Слишком большое окно (p = 14) не даёт выигрыша в точности, потому что избыточные входы коррелированы и не добавляют информации о динамике; зато резко ускоряет сходимость (32 эпохи против 115 при p = 7), так как задача становится лучше обусловленной.
2. Влияние числа скрытых нейронов N. С ростом N от 1 до 8 скорость обучения слегка растёт (129 → 101 эпоха), тестовая ошибка слабо снижается (0.0221 → 0.0210). Переобучения нет: train и test MAE совпадают с точностью до разброса по seed (разница < 0.001). Минимального N = 1 достаточно для воспроизведения суммы двух гармоник на одношаговом горизонте, одного сигмоидного нейрона хватает, чтобы сформировать нелинейную комбинацию входов; дополнительные нейроны дают лишь маргинальное улучшение.
3. Почему на выходе линейная функция. Задача прогнозирования - регрессия на всей числовой оси; сигмоида на выходе ограничила бы прогноз интервалом (0; 1) (в исходной шкале после денормализации - диапазоном train), сделала бы невозможным выход за пределы обучающего диапазона и внесла бы насыщение, обнуляющее градиент. При сигмоиде на выходе сеть не смогла бы предсказывать значения за пределами [x_min; x_max] и обучалась бы крайне медленно из-за насыщения. Обучение на ненормализованных значениях (порядка ±1.3) при сигмоидном скрытом слое привело бы к насыщению нейронов уже на первых эпохах, взрыву/затуханию градиентов и отсутствию сходимости.
4. Почему нормализация только по train. Иначе тестовая информация попадает в подготовку данных (утечка), оценка качества завышается и перестаёт быть честной. В нашем случае тестовые нормированные значения лежат в [0.011; 0.992], формально внутри [0; 1], потому что тестовый участок не выходит за пределы train-диапазона. Если бы тест выходил за [0; 1], сигмоиды скрытого слоя насыщались бы на «крайних» входах, градиенты обнулялись бы, и ошибка на таких точках росла бы скачком.
5. Устойчивость к инициализации. Обучение устойчиво: во всех сериях по 5 seed конфигурации p ∈ {4, 7, 14} и N ∈ {1, 2, 4, 8} достигали Ee, разброс числа эпох - 4-14 % от среднего, разброс test MAE - единицы процентов. Единственные «провалы» - p = 2 (5 из 5 запусков не достигли Ee за 20 000 эпох): при слишком малом окне задача неразрешима при любом seed. Зависимость: провалы определяются прежде всего p (информативностью входа), затем η (при η = 0.01 нужно ~1250 эпох, при 0.3 - 48) и в меньшей степени N.
6. Рост ошибки рекурсивного прогноза. Ошибка накапливается: каждое предсказание становится входом следующего шага, и погрешность модели умножается на чувствительность сети к входу; в окрестности экстремумов, где сигмоиды насыщены, эта чувствительность максимальна. На первых 6-7 шагах ошибка близка к одношаговой (0.007-0.022), на 8-м шаге превышает 5 % размаха (0.134), к 26-му достигает 0.47. Одношаговый прогноз на том же участке точен (MAE = 0.021), многошаговый MAE за 30 шагов = 0.146, в 7 раз хуже. Прогноз пригоден примерно до 7 шагов.
7. Обобщающая способность и меры улучшения. Различие train/test MAE мало (0.0222 против 0.0224), sMAPE выше на тесте (8.6 % против 15.6 %) из-за точек вблизи нуля, а не из-за переобучения. Обобщение хорошее: модель не запоминает train. Улучшить его могли бы: ранняя остановка по валидационной выборке (здесь не требуется, переобучения нет), уменьшение N (выигрыша не даёт), добавление шума, увеличение обучающей выборки. Эксперимент с шумом (σ = 0.05 в норм. шкале только на train) показал: модель перестаёт достигать Ee за 20 000 эпох (5 из 5 запусков), test MAE на чистых данных вырастает в ~5 раз (0.1165 против 0.0222) при std 0.044, сеть начинает аппроксимировать шум, и обобщающая способность падает. Это подтверждает, что чистота обучающих данных для данной архитектуры критична, а шумовая аугментация в таком виде не помогает, а вредит.

Вывод: на основе многослойного персептрона p–N–1 решила задачу прогнозирования временного ряда; научилась формировать обучающую выборку методом скользящего окна, нормализовала данные, оценила качество одношагового и многошагового прогноза и исследовала влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.